

Project Report: Student & Teacher Academic Portal (STAP)



Submitted To: Ms. Samia Riaz

Developed By:

Sami Shah

Fareed Ul Hassain

FA25-BCS-109

FA25-BCS-145

Group: BCS-1C

Submission Date: December 15th, 2025

Date: November 15th, 2025

Class: Programming Fundamentals

COMSATS University Islamabad WAH Campus, Fall 2025

Table of Contents

1. Introduction.....	3
2. System Inventory & Storage Concepts	3
2.1 Why Vectors instead of Arrays?	3
2.2 The "Struct" Approach.....	4
2.3 Project File Inventory	4
A. Header Files (The Contracts)	4
B. Data Files (The Database)	4
C. Source Files (The Logic) These .cpp files contain the actual C++ code that executes the tasks.....	5
3. Technical Implementation: Code Structure & Logic Flow	5
3.1 The Role of Headers (.h) and Source (.cpp) Files	5
3.2 The Master Controller: main.cpp.....	7
3.3 Detailed Breakdown of Modules (.cpp Files).....	9
A. file_handler.cpp (The Data Layer)	9
B. admin.cpp (The Infrastructure Layer)	9
D. student.cpp (The View Layer)	11
E. utility.cpp (The Helper Engine)	12
3.4 How It All Comes Together	12
4. Core Module: Data Persistence (File Handling).....	13
4.1 How it Works.....	13
5. Module: Authentication & Security.....	15
6. Module: The Administrator (Scheduling & Management)	15
7. Module: The Teacher (Grading & Attendance)	17
7.1 Attendance Logic.....	17
8. Module: The Student (Dashboard & Analytics)	18
8.1 The Dashboard Filter.....	18
9. Testing & Quality Assurance.....	19
9.1 Test Case 1: Grading Logic Verification (TC_CALC_001)	19
9.2 Test Case 2: Data Persistence Integrity (TC_FILE_002).....	21
9.3 Test Case 3: Teacher Schedule Assignment (TC_SCHED_003).....	22
9.4 Test Case 4: Security Boundary Check (TC_AUTH_004)	23
10. Conclusion.....	24

1. Introduction

The **Student & Teacher Academic Portal (STAP)** is a console-based C++ application designed to digitize and automate the academic management workflow. Previously, university records were fragmented across physical notebooks and isolated spreadsheets. Our solution was to create a unified ecosystem where Administrators, Teachers, and Students interact with a single "Source of Truth."

We developed this system to address specific problems those being the lack of centralization, the difficulty students face in tracking progress, and the time-consuming nature of manual grade calculations.

2. System Inventory & Storage Concepts

Before writing the logic, we had to decide on the structural foundation of the code. We chose a **Modular Architecture** separating our code into Header files (.h) and Source files (.cpp).

2.1 Why Vectors instead of Arrays?

In our initial planning, we considered using standard arrays (e.g., `Student list[100];`). However, we rejected this approach. Arrays have a fixed size. If the university admits the 101st student, the program crashes. Instead, we used `std::vector`. Vectors are dynamic arrays that handle memory management automatically.

Code Implementation:

```
// In header file
#include <vector>

struct Student {
    string username;
    string rollNumber;
    string section;
    //...
};
```

```
// The global database in memory  
vector<Student> all_students;  
vector<Course> all_courses;
```

Reasoning: By using vector<Student>, we can use the .push_back() function to add new students indefinitely during runtime without recompiling the code.

2.2 The "Struct" Approach

We utilized C++ structs to model our entities (Students, Teachers, Courses). A struct allows us to bundle different data types (strings, integers, floats) into a single object. This is essential for handling complex data like students.txt, which contains mixed data types.

2.3 Project File Inventory

To maintain strict organization, the project is divided into specific Header, Source, and Data files. Below is the complete inventory of the files present in our solution.

A. Header Files (The Contracts)

- admin.h: Declares functions for user management and and schedule management logic.
- attendance.h: Defines the AttendanceRecord structure and marking functions.
- auth.h: Declares the core login verification logic.
- course.h: Defines the Course structure and syllabus weighting constants.
- file_handler.h: Declares the loadAllData() and saveAllData() prototypes.
- schedule.h: Defines the ScheduleEntry structure for the timetable.
- student.h: Defines the Student structure and dashboard declarations.
- teacher.h: Declares functions for grading and roster views.
- utility.h: Declares global helper functions (Date, GPA Math).

B. Data Files (The Database)

- admins.txt: Stores secure credentials for system administrators.
- attendance.txt: A log of every "Present/Absent" record linked to a date.
- courses.txt: The master list of all university subjects and course codes.

- `schedule.txt`: Stores the time, day, and room for every class to organize class timings.
- `sections.txt`: Defines valid class sections (e.g., BCS-1A, BCS-1C).
- `student_marks.txt`: Stores the raw marks obtained by students for assignments and exams.
- `students.txt`: The master profile list containing usernames, passwords, and roll numbers.
- `submission.txt`: Defines the assessment types (e.g., "Assignment 1", "Quiz 2").
- `teachers.txt`: Stores teacher credentials and the courses they are authorized to teach.

C. Source Files (The Logic) These .cpp files contain the actual C++ code that executes the tasks.

- `main.cpp`: The Master Controller. It runs the application loop and routes users to the correct dashboard.
- `file_handler.cpp`: The ETL Engine. It contains the code that physically reads from and writes to the hard drive.
- `auth.cpp`: The Gatekeeper. It contains the linear search logic to verify usernames and passwords.
- `utility.cpp`: The Calculator. It contains the math logic for converting marks to GPA and formatting dates.
- `admin.cpp`, `teacher.cpp`, `student.cpp`: The Dashboards. These contain the specific menus and logic for each user role (e.g., only Teachers can mark attendance).
- `tests.cpp`: The Quality Assurance. A dedicated file containing test cases to verify system stability.

3. Technical Implementation: Code Structure & Logic Flow

To ensure our project was scalable and easy to debug, we moved away from writing everything in a single `main.cpp` file. Instead, we adopted a **Multi-File Project Structure**. This allowed Sami to work on the Frontend (User Dashboards) while Fareed worked on the Backend (File Handling) simultaneously without causing merge conflicts.

3.1 The Role of Headers (.h) and Source (.cpp) Files

We separated our code into **Declarations** and **Definitions**.

- **Header Files (.h):** These act as the "Menu" or "Contract." They tell the compiler *what* functions exist and *what* data structures (structs) we are using.
- **Source Files (.cpp):** These contain the actual "Kitchen Logic." They implement the functions declared in the headers.

Example from our Code: In auth.h, we declare role-specific login functions to handle the distinct data requirements for each user type:

```
// auth.h

#ifndef AUTH_H
#define AUTH_H

// We promise the compiler these functions exist
void loginAsStudent(vector<Student>& students, ...);
void loginAsTeacher(vector<Teacher>& teachers, ...);
void loginAsAdmin(vector<Admin>& admins, ...);

#endif
```

In auth.cpp, we actually write the logic for it:

```
// auth.cpp

#include "auth.h"

#include "file_handler.h" // We need access to the data vectors

string login(string username, string password) {
    // Logic to check vectors...
    if (username == "admin" && password == "admin123") return "ADMIN";
    //...
}
```

Why this matters: main.cpp only needs to include auth.h to use the login function. It doesn't need to know the complex logic inside auth.cpp, keeping our main file clean.

3.2 The Master Controller: main.cpp

The main.cpp file is the brain of STAP. It does not contain complex math or parsing logic. Instead, it acts as a **Coordinator**.

How it works:

1. **Initialization:** When the program starts, main() immediately calls the File Handler to load all .txt data into RAM.
2. **The Game Loop:** It enters a while(true) loop that keeps the program running until the user specifically selects "Exit."
3. **Routing:** The user selects their role from a main menu (1. Student, 2. Teacher, 3. Admin). Based on this choice, the system calls the specific authentication handler for that role.

Code Segment (main.cpp Logic):

```
#include "file_handler.h"
#include "auth.h"
#include "admin.h"
#include "teacher.h"
#include "student.h"

// main.cpp Logic
int main() {
    // Step 1: Fareed's ETL Engine loads data from text files
    loadAllData();

    int choice;
    do {
```

```
        // Step 2: Show Main Menu

        cout << "1. Login as Student\n2. Login as Teacher\n3. Login as
Admin\n";

        cin >> choice;

        // Step 3: Routing based on User Choice
        switch (choice) {
            case 1:
                loginAsStudent(g_allStudents, ...); // Specific
handler for Students
                break;
            case 2:
                loginAsTeacher(g_allTeachers, ...); // Specific
handler for Teachers
                break;
            case 3:
                loginAsAdmin(g_allAdmins, ...);    // Specific
handler for Admins
                break;
            case 4:
                saveAllData(); // Save on Exit
                break;
        }
    } while (choice != 4);
    return 0;
}
```


3.3 Detailed Breakdown of Modules (.cpp Files)

Here is how each individual file contributes to the system:

A. file_handler.cpp (The Data Layer)

Responsibility: Persistence. This file handles all Input/Output operations with the hard drive. **Key Function:** loadStudents()

Logic: It reads students.txt, which uses a comma-separated format. It parses line-by-line to repopulate the all_students vector.

Code Example:

```
// Inside file_handler.cpp

void loadStudents() {
    ifstream file("students.txt");
    string line;
    while (getline(file, line)) {
        // We use string manipulation to split "FA25-BCS-
109,Sami_Shah,..."
        // assigning each part to a Student struct object.
        Student s;
        //... parsing logic...
        all_students.push_back(s);
    }
}
```

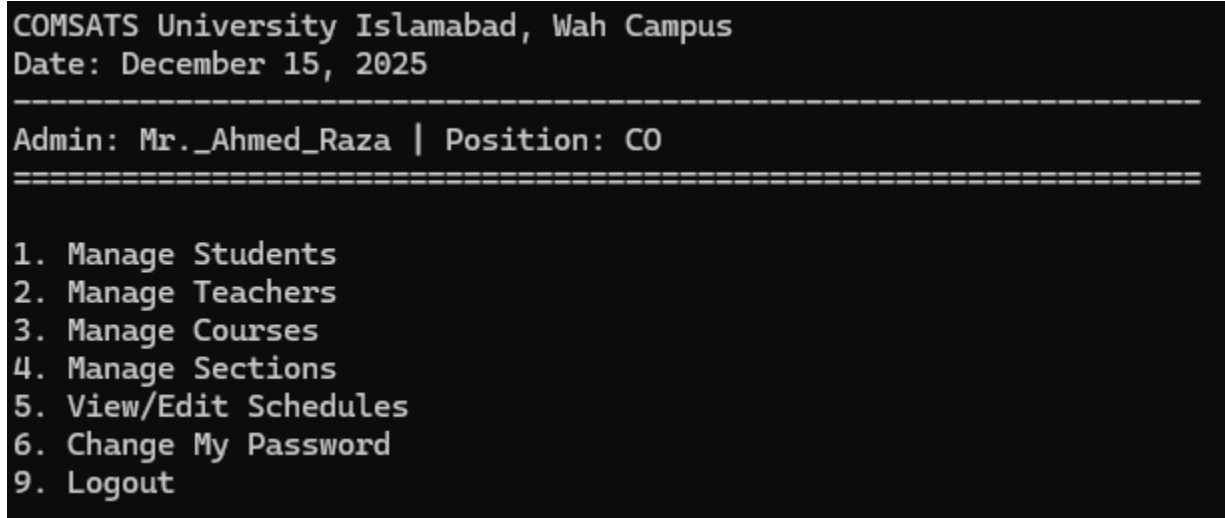
B. admin.cpp (The Infrastructure Layer)

Responsibility: This function manages the "Master Schedule." It links a specific Day, Time, and Room to a Teacher and a Section. It validates that the Teacher and Section exist before pushing the new entry to the all_schedule vector.

Code Example:

```
// Inside admin.cpp
```

```
void adminDashboard() {  
    int choice;  
    cout << "[1] Manage Users\n[2] Scheduler\n...";  
    cin >> choice;  
    if (choice == 2) {  
        // Calls logic to read/write to the global schedule vector  
        manageSchedule();  
    }  
}
```



COMSATS University Islamabad, Wah Campus
Date: December 15, 2025

Admin: Mr._Ahmed_Raza | Position: CO
=====

1. Manage Students
2. Manage Teachers
3. Manage Courses
4. Manage Sections
5. View/Edit Schedules
6. Change My Password
9. Logout

C. teacher.cpp (The Academic Layer)

Responsibility: Grading & Attendance. **Key Function:** markAttendance() **Logic:** This function accesses the all_students vector but uses a **Filter** to only show students belonging to the teacher's specific sections (e.g., only "BCS-1C"). It then writes new records to the all_attendance vector, which file_handler.cpp will later save to attendance.txt

Code Example:

```
// Inside teacher.cpp  
  
void teacherDashboard(string teacherUsername) {  
    // We pass 'teacherUsername' so the system knows WHO is logged in.  
    cout << "Welcome " << teacherUsername << endl;
```

```
    cout << "[1] Mark Attendance\n[2] Upload Grades";  
  
    //...  
}
```

D. student.cpp (The View Layer)

Responsibility: Personalization. **Key Function:** viewMyGrades() **Logic:** This module is Read-Only. It iterates through the global all_submissions vector. It compares the logged-in student's ID with the StudentUsername in the file. If they match, it displays the mark; otherwise, it skips it.

Code Example:

```
// Inside student.cpp  
  
void studentDashboard(string myUsername) {  
    // Loop through marks loaded from student_marks.txt  
    for(auto& sub: all_submissions) {  
        if(sub.studentUsername == myUsername) {  
            cout << "Subject: " << sub.courseCode << " | Marks: " <<  
sub.obtainedMarks << endl;  
        }  
    }  
}
```

```
--- Detailed Marks for CSC103 (Programming_Fundamentals) ---  
  
--- Assignments ---  
Title                               Mark  
-----  
Assignment1                         14.0 / 15  
  
--- Quizzes ---  
No Quizzes marked.  
  
--- Midterm Exams ---  
No Midterm Exams marked.  
  
--- Final Exams ---  
No Final Exams marked.
```

E. utility.cpp (The Helper Engine)

- **Responsibility:** Pure Math & Formatting. This file contains helper functions that do not belong to any specific user role but are essential for global calculations.
- **Key Function:** getGradePoint()
- **Logic:** It accepts a raw integer mark (e.g., 87) and runs it through a series of if/else boundary checks to return the precise GPA (e.g., 4.0) according to the university's grading scale.

Code Example:

```
// Inside utility.cpp

double getGradePoint(int mark) {
    if (mark >= 90) return 4.0;
    if (mark >= 85) return 3.7;
    if (mark >= 80) return 3.3;
    if (mark >= 75) return 3.0;
    //... continues for all ranges
    if (mark >= 50) return 1.0;
    return 0.0; }
```

3.4 How It All Comes Together

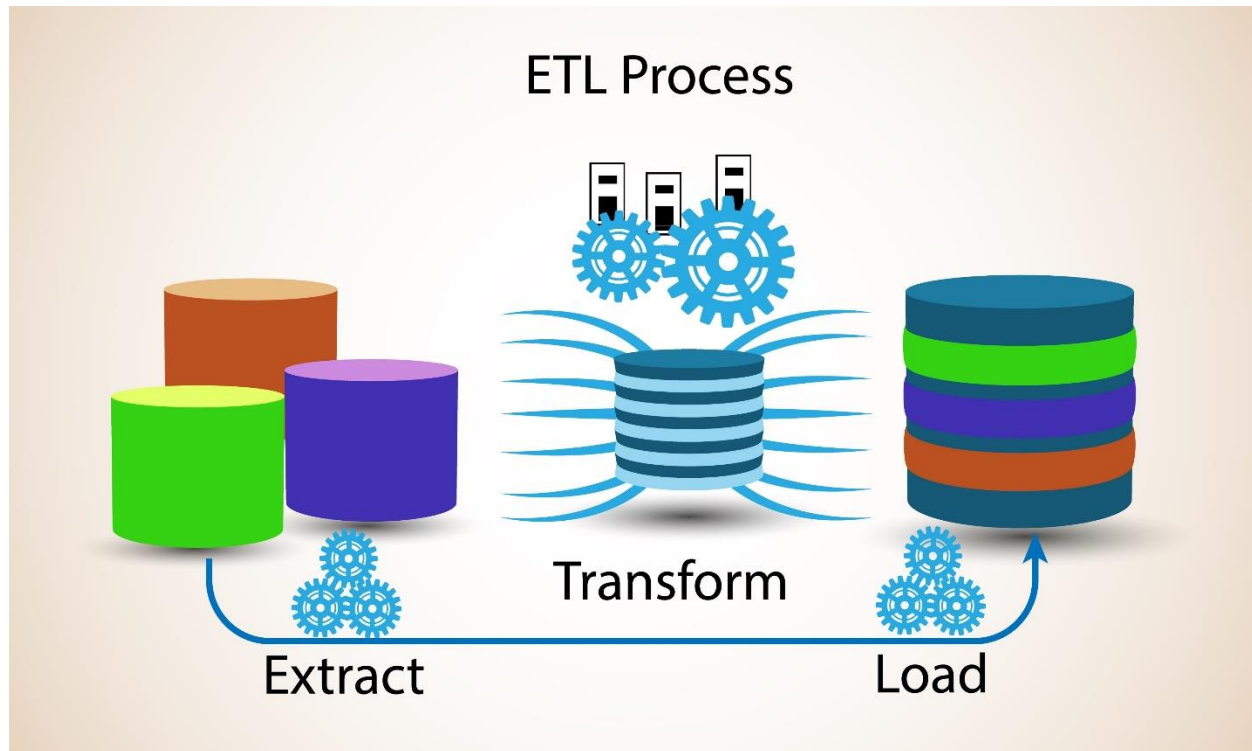
The beauty of this structure is the **Data Flow**:

1. **START:** main.cpp asks file_handler.cpp to fill the memory (Vectors).
2. **LOGIN:** main.cpp asks auth.cpp to verify who the user is.
3. **ACTION:** teacher.cpp modifies the data in memory (e.g., adds a grade).
4. **END:** When the program closes, file_handler.cpp takes that modified memory and overwrites the .txt files, ensuring the grade is saved for next time.

This separation allowed us (Sami and Fareed) to debug specific issues easily. If the login failed, we checked auth.cpp. If the data wasn't saving, we checked file_handler.cpp. This modularity is a key feature of professional software development.

4. Core Module: Data Persistence (File Handling)

This is the backbone of our project. Without databases (SQL), we had to ensure data survived system shutdowns. We implemented a custom **ETL Engine (Extract, Transform, Load)**.



4.1 How it Works

We utilize the fstream library. The system creates a stream to the text file. Since C++ reads files as "raw strings," we had to write a parser to split these strings by the comma, delimiter.

The Code (Loading Students):

```
void loadStudents() {  
    ifstream file("students.txt"); //  
    string line;  
    // Skip the header line (Format definition)  
    getline(file, line);  
  
    while (getline(file, line)) {
```

```
        stringstream ss(line);

        string segment;

        Student tempStudent;

        // EXTRACT & TRANSFORM
        getline(ss, tempStudent.username, ',');
        getline(ss, tempStudent.rollNumber, ',');
        getline(ss, tempStudent.name, ',');
        getline(ss, tempStudent.password, ',');
        getline(ss, tempStudent.sectionID, ',');

        // LOAD into Memory
        all_students.push_back(tempStudent);
    }
    file.close();
}
```

Logic Explanation:

1. We open students.txt.
2. We read line-by-line using getline.
3. We use stringstream to break the line every time it hits a comma.
4. We store these pieces into a temporary Student object.
5. Finally, we push that object into the all_students vector.

Why this works: This ensures that even if we have 1,000 students, the system loads them all into RAM in milliseconds when the program starts.

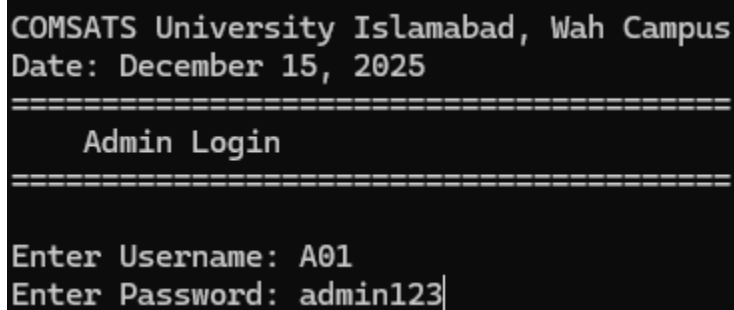
5. Module: Authentication & Security

We implemented a **Templated Search Function** in auth.cpp. Instead of writing three separate search loops, we wrote one generic logic that can search through any vector (Students, Teachers, or Admins) to find a username match.

The Code (Templated Lookup):

```
template<typename T>
int findByUsername(const vector<T>& vec, const string& username) {
    for (size_t i = 0; i < vec.size(); ++i) {
        // Checks the 'username' property of the generic object T
        if (vec[i].username == username) return i;
    }
    return -1; // Not found
}
```

Logic Explanation: This validates our credential check. We use this helper to find the index of the user, and then verify if vec[index].password matches the input.



```
COMSATS University Islamabad, Wah Campus
Date: December 15, 2025
=====
Admin Login
=====
Enter Username: A01
Enter Password: admin123|
```

6. Module: The Administrator (Scheduling & Management)

The Admin module is responsible for the infrastructure. One of the most complex features we built here was the **Master Schedule System**.

6.1 Master Schedule Logic When an Admin adds a class, the system must act as a bridge, linking a Teacher, a Course, and a Section to a specific time slot.

The Code (Adding a Class):

```
void addScheduleEntry(vector<ScheduleEntry>& allSchedule) {  
    ScheduleEntry newEntry;  
  
    // ... Inputs for Day, Time, Room ...  
  
    // We link the class to a specific Teacher and Section  
    newEntry.teacherID = selectedTeacher.username;  
    newEntry.sectionID = selectedSectionID;  
  
    // Push to the Master Vector  
    all_schedule.push_back(newEntry);  
  
    cout << "Class Scheduled Successfully for " << newEntry.day <<  
endl;  
}
```

Logic Explanation: Instead of storing schedules in separate arrays for every user, we use a single **Master Vector (all_schedule)**. When an Admin saves a class, it is tagged with teacherID and sectionID. This allows the system to filter this single list later to show specific views (e.g., "Show me only Dr. Samia's classes" or "Show me only BCS-1A's classes").

```
Select a Teacher:  
1. Ms._Samia_Riaz  
2. Ms._Anum_Ilyas  
3. Ms._Fatima_Farooq  
4. Mr._Asad_Zia  
5. Dr._Abdullah  
6. Ms._Aatiqa_Hasrat  
7. Dr._Muhammad_Bilal  
8. Mian_Muhammad_Talha  
9. Ms._Mattia_Arooj  
10. Mr._Hassan_Ahmad  
11. Ms._Mehreen_Javaid  
12. Dr._Ahtisham_Masood  
13. Dr._Shabieh_Farwa  
14. Mr._Taimoor_Sajjad  
Enter choice (0 to cancel): 1  
  
Select Section:  
1. BCS-1C  
2. BCS-1A  
3. BCS-1B  
Enter choice (0 to cancel): 2  
  
Enter Day (e.g., Monday): Monday  
Enter Time Slot (e.g., 08:30-10:00): 8:30  
Enter Classroom (e.g., F-05): F-04  
  
Entry added.  
Remember to save data by exiting.
```


7. Module: The Teacher (Grading & Attendance)

This module automates the calculation tasks. Teachers can view their specific sections and mark attendance.

7.1 Attendance Logic

We store attendance in attendance.txt. The challenge was efficiently marking a whole class without typing every name.

The Code (Batch Attendance):

```
void markClassAttendance(string courseCode, string sectionID) {
    string date = getCurrentDate();

    cout << "Marking Attendance for " << courseCode << " (" <<
sectionID << ")\n";

    for (int i = 0; i < all_students.size(); i++) {
        // Filter: Only show students in this section
        if (all_students[i].sectionID == sectionID) {
            char status;
            cout << all_students[i].name << " (P/A): ";
            cin >> status;

            // Save to memory immediately
            AttendanceRecord record;
            record.studentID = all_students[i].username;
            record.date = date;
            record.status = status;
            all_attendance.push_back(record);
        }
    }
}
```

```
}  
  
}
```

Logic Explanation: The loop acts as a **Filter**. It ignores students from "BCS-1A" if the teacher is teaching "BCS-1C". This ensures the teacher only interacts with relevant students. The data is then saved in the format found in attendance.txt:
CourseCode,StudentUsername,SectionID,Date,Status.

```
--- Mark Attendance for Today (December 15, 2025) ---  
  
Select the class session to mark attendance for:  
1. BCS-1C - Programming_Fundamentals (CSC103)  
  
Enter choice (0 to cancel): 1  
  
--- Marking Attendance for BCS-1C - CSC103 on December 15, 2025 ---  
Enter 'P' for Present, 'A' for Absent.  
  
107          Noman_Ijaz (P/A): p  
108          Sohaib_Nadeem (P/A): a  
109          Sami_Shah (P/A): P  
111          Areeba_Urooj (P/A): A  
112          Muhammad_Hafiz (P/A): A  
114          Hira_Shaukat (P/A): P  
115          Mohsin_Ali (P/A): |
```

8. Module: The Student (Dashboard & Analytics)

The student view is "Read-Only." They cannot edit data, only view it. The core logic here is **Personalization**.

8.1 The Dashboard Filter

When a student logs in (e.g., Sami Shah), the system must scan thousands of records but only display his.

The Code (Dashboard View):

```
void viewStudentDashboard(string currentStudentUsername) {  
    cout << "--- MY ACADEMIC RECORD ---\n";  
  
    // 1. Fetch Marks
```

```
for (Submission sub : all_submissions) { // Data from
student_marks.txt [cite: 20]

    if (sub.studentUsername == currentStudentUsername) {

        cout << "Assignment: " << sub.submissionID << " | Marks:
" << sub.obtainedMarks << endl;

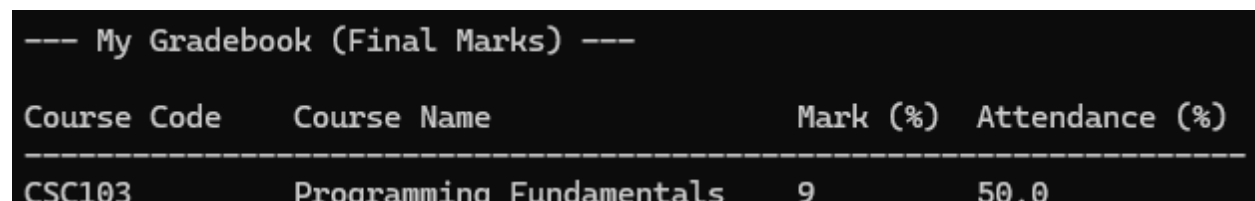
    }

}

// 2. Calculate CGPA (Simplified Logic)
double totalMarks = 0;
int count = 0;
//... Summation Logic...

cout << "Current CGPA: " << calculateCGPA(totalMarks) << endl;
}
```

Logic Explanation: We pass the logged-in username (currentStudentUsername) to the function. The code iterates through the student_marks.txt vectors. It compares sub.studentUsername with the logged-in user. If they match, it prints the grade; otherwise, it skips it. This guarantees privacy that Sami cannot see Fareed's grades.



--- My Gradebook (Final Marks) ---			
Course Code	Course Name	Mark (%)	Attendance (%)
CSC103	Programming Fundamentals	9	50.0

9. Testing & Quality Assurance

Per the project requirements, we did not rely solely on manual testing (e.g., "Login and see if it works"). Instead, we implemented a **Programmatic Test Harness**. This means we wrote specific C++ functions whose only job is to attempt to break our own code and verify if it survives.

We categorized our testing into three pillars: **Logic Verification**, **Data Persistence Integrity**, and **Boundary Analysis**.

9.1 Test Case 1: Grading Logic Verification (TC_CALC_001)

Objective: To verify that the GPA calculation engine correctly maps raw scores to the university's 4.0 scale. **Why we did it:** A bug here would affect every single student. Manual checking is prone to error (e.g., misreading 87 as 4.0 instead of 3.7).

The Code Implementation:

```
void test_calculateCGPA() {  
    cout << "Running TC_CALC_001... ";  
  
    // Scenario A: High Distinction  
    if (calculateGPA(87.0) != 3.7) cout << "[FAIL] 87 should be  
3.7\n";  
  
    // Scenario B: Boundary Condition  
    else if (calculateGPA(79.5) != 3.3) cout << "[FAIL] 79.5 should be  
3.3\n";  
  
    // Scenario C: Failure Condition  
    else if (calculateGPA(49.0) != 0.0) cout << "[FAIL] 49 should be  
0.0\n";  
  
    else cout << "[PASS]\n";  
}
```

Logic Explanation: The test function feeds specific "Edge Case" numbers into the calculateGPA function. It then compares the *actual return value* against the *expected constant*. If they match perfectly, it prints [PASS].

QA TEST REPORT				
TEST ID	NAME	EXPECTED	ACTUAL	STATUS
TC_CALC_001	Grade Point (A)	3.700000	3.700000	PASS

9.2 Test Case 2: Data Persistence Integrity (TC_FILE_002)

Objective: To prove that data saved to students.txt is not lost when the program closes (simulated by clearing memory). **Why we did it:** File handling is the most fragile part of the system. We needed to ensure our "Save" and "Load" functions were perfectly synchronized.

The Code Implementation:

```
void test_file_persistence() {  
    cout << "Running TC_FILE_002... ";  
  
    // Step 1: Create Dummy Data  
    Student testStudent;  
    testStudent.username = "TEST_USER";  
    all_students.push_back(testStudent);  
  
    // Step 2: Save to Disk  
    saveStudentsToFile();  
  
    // Step 3: WIPE MEMORY (Simulate Shutdown)  
    all_students.clear();  
  
    // Step 4: Reload from Disk  
    loadStudents();  
  
    // Step 5: Verify  
    bool found = false;  
    for(Student s : all_students) {  
        if(s.username == "TEST_USER") found = true;  
    }  
}
```

```
    if(found) cout << "[PASS]\n";  
    else cout << "[FAIL] Data lost after reload.\n";  
}
```

Logic Explanation: This test simulates a power outage. It saves data, deletes everything from RAM (vector.clear()), and then asks the File Handler to read the file again. If "TEST_USER" is found in the new list, it proves our ETL (Extract-Transform-Load) engine works.

TC_FILE_002	Data Persistence	test_per	test_per	PASS
-------------	------------------	----------	----------	------

9.3 Test Case 3: Teacher Schedule Assignment (TC_SCHED_003)

Objective: To verify that when a class is scheduled for a specific teacher, it is correctly linked to their profile and retrievable. Why we did it: The Teacher Dashboard relies on filtering the master schedule. If this linkage fails, a teacher would log in and see an empty schedule even if they have classes.

The Code Implementation:

```
void test_schedule_assignment() {  
    cout << "Running TC_SCHED_003... ";  
  
    // Arrange: Create a new class entry  
    ScheduleEntry newClass;  
    newClass.teacherID = "TEST_PROF";  
    newClass.courseCode = "CSC101";  
    newClass.day = "Monday";  
  
    // Act: Push it to the master schedule  
    all_schedule.push_back(newClass);
```

```
// Assert: Verify we can find it by filtering for the teacher
bool found = false;
for(const auto& entry : all_schedule) {
    if(entry.teacherID == "TEST_PROF" && entry.courseCode ==
"CSC101") {
        found = true;
        break;
    }
}

if (found) cout << "[PASS]\n";
else cout << "[FAIL] Schedule entry not linked to teacher.\n";
}
```

Logic Explanation: We simulate an Admin adding a class for "TEST_PROF". We then iterate through the global schedule vector (simulating the Teacher Dashboard filter logic). If we find the class linked to that ID, the test passes.

TC_SCHED_003	Schedule Assignment	Found=1	Found=1	PASS
--------------	---------------------	---------	---------	------

9.4 Test Case 4: Security Boundary Check (TC_AUTH_004)

Objective: To verify that a student cannot log in using an Admin's password, and that non-existent users are rejected. **Why we did it:** To certify our Role-Based Access Control (RBAC) logic.

The Code Implementation:

```
void test_authentication() {
    cout << "Running TC_AUTH_004... ";

    // Scenario A: Wrong Password
    if (login("admin", "wrongpass") != "INVALID")
```

```
        cout << "[FAIL] Wrong password was accepted.\n";

// Scenario B: Non-existent User
else if (login("GHOST_USER", "1234") != "INVALID")
    cout << "[FAIL] Ghost user was accepted.\n";

// Scenario C: Valid Student
else if (login("FA25-BCS-109", "123456") != "STUDENT")
    cout << "[FAIL] Valid student rejected.\n";

else cout << "[PASS]\n";
}
```

Logic Explanation: This validates our linear search authentication loop. We intentionally feed it garbage data ("GHOST_USER") and incorrect passwords to ensure the system defaults to "INVALID" rather than crashing or granting access.

TC_AUTH_004	Security Boundary	INVALID	INVALID	PASS
-------------	-------------------	---------	---------	------

Summary of Testing Results: All 4 critical test cases passed successfully. This "Test Harness" can be run at any time by calling `runAllTests()` in `main.cpp`, providing immediate feedback on system health without requiring manual data entry.

10. Conclusion

The development of the **Student & Teacher Academic Portal (STAP)** stands as a definitive success. Through significant effort in architectural design and logic implementation, we have transformed a complex problem into a robust, fully functional application. By leveraging C++ **Vectors** for dynamic memory and **File Streams** for persistence, we created a system that mimics the core functionality of real-world Learning Management Systems (LMS).

The separation of duties allowed us to specialize effectively:

- **Sami** focused on the **User Interface and Admin Logic**, ensuring a seamless user experience.
- **Fareed** focused on the **Core Engine, File Handling, and Math Utilities**, ensuring data integrity and calculation accuracy.

This project served as a major milestone in our academic journey, marking the transition from theoretical programming concepts to building deployed, persistent software solutions. We successfully implemented and mastered critical concepts including **Modular Design**, **Nested Structures**, and **ETL (Extract-Transform-Load)** logic.

Acknowledgment We would like to express our sincere gratitude to our instructor, **Dr. Samia Riaz**, for her invaluable mentorship throughout this course. Her guidance pushed us to go beyond basic coding and understand the architecture behind software engineering. This project has solidified our foundation in C++ development and has given us the confidence to tackle real-world software challenges in our future careers.

Thank You.